Section 3.5

EstimatedRTT = 1-alpha(EstimatedRTT) + alpha(SampleRTT)
= 0.875(EstimatedRTT) + 0.125(SampleRTT)
(alpha is recommended to be 0.125)

DevRTT = 1+beta(DevRTT + beta) * |SampleRTT - EstimatedRTT|
= 0.75(DevRTT + 0.25) * |SampleRTT - EstimatedRTT|
= (beta is recommended to be 0.25)

So, TimeoutInterval = EstimatedRTT + 4 * DevRTT

Section 3.6

LastByteSent - LastByteAcked <= min{cwnd, rwnd}

where cwnd is congestion window and rwnd is receive window

Section 3.7

- TCP is self-clocking → successful ACKs increase window size
- A lost segment implies congestion
    - -Ex: Receipt of four ACKs (one original and three dups) is interpreted as a loss event
    - -Congestion window size should decrease
- An ACKed segment indicates all is well and the sender's rate can be increased when an ACK arrives for a previously unACKed segment
    - -Congestion window size should increase
- TCP Congestion-Control Algorithm:
    - Slow start
        - When cwnd is initialized, it's value is 1 MSS (maximum segment size)
            - sending rate = mss/rtt
        - Increases by 1 MSS for every first ACK for transmitted segments
        - If loss event (timeout) occurs, cwnd is set to 1 mss and starts over
            - Also sets ssthresh (slow start threshold) to cwnd/2
        - Or if cwnd == ssthresh, slow start ends and TCP goes into congestion avoidance mode
        - Or if three dup ACKs are detected → TCP performs fast retransmit and enters fast recovery state
    - Congestion avoidance
        - value of cwnd is approximately half its value when congestion was last encountered
        - cwnd increased by 1 MSS every RTT
        - If loss event (timeout) occurs, cwnd is set to half + 3 mss
            - ssthresh is cwnd/2
            - fast recovery is then entered
    - Fast recovery

        - value of cwnd is increased by 1 mss for every dup ack received for the missing segment that caused TCP to enter fast recovery
        - If timeout occurs, fast recovery transitions to slow start but before that:
            - cwnd is set to 1 mss
            - ssthresh is cwnd/2
- TCP Tahoe
    - Unconditionally cut its cwnd to 1 mss and entered slow-start after either timeout or triple-dup ACK
    - Ex:
        - When a less event occurs at 12 MSS:
            - ssthresh = 0.5 * cwnd = 6 MSS
            - cwnd = 1 MSS
            - cwnd grows exponentially (doubles every time) until it reaches ssthresh, at which point it grows linearly (one every time)
- TCP Reno
    - Incorporates fast recovery instead of above what Tahoe does
    - Ex:
        - When a loss event occurs at 12 MSS:
            - ssthresh = 0.5 * cwnd = 6 MSS
            - cwnd = cwnd / 2 + 3 = 9 MSS
            - cwnd grows linearly

Section 4.3

- The maximum amount of data that a link-layer frame can carry is called the maximum transmission unit (MTU)

Section 5.1

- Dijkstra's

Section 5.3

- OSPF

Section 5.4

- BPF

Section 6.3.2

- ALOHA
- CSMA
- CSMA/CD